

Name: Arnav Krishnan

Report Date: Week of September 14

Work Completion Conflicts: Missed portion of Friday due to club fair- had to move the drone & rc plane and put away the poster. Also, my lead had to cover for CAD, so I didn't really have as much interaction with my lead, I did a lot of debugging myself.

Completed Tasks

Learning and downloading software (Monday)

The image displays two screenshots. The top screenshot shows the Phoenix Tuner X software interface, which includes a sidebar with navigation options like 'Requirements', 'Migration', 'Phoenix & Features', 'API Migration', 'CAN Bus Utilization', 'Phoenix Tuner X', 'Connecting Tuner', 'Device List', 'Tuner History', 'Device Details', 'Tuner Configs', 'Self Test Snapshot', 'Controlling Devices', 'Pinning', 'Multi-device Plot & Control', 'Profile Page', 'Pigpen 2.0 Calibration', 'Tools', 'Swarm Project Generator', and 'Tuner Elevator Generator'. The main content area shows a 'Phoenix Tuner X' title and a description of the application. The bottom screenshot shows the REV Hardware Client 2 download page, featuring the REV logo and two large green buttons labeled 'DOWNLOAD FOR INTEL MAC' and 'DOWNLOAD FOR APPLE SILICON MAC'.

Monday was mainly a day of preparation.

Before we used the testboard, I needed all the tools for section 0 of the Code Intro installed. I also needed to touch up on Java, as I haven't used it much since my AP Comp Sci days...

The testboard has a REV power distribution hub, a Kraken x60 with a TalonFX controller and creative encoder, a RoboRIO, a wifi radio; e.g., each piece of hardware will correspond to a piece of software.

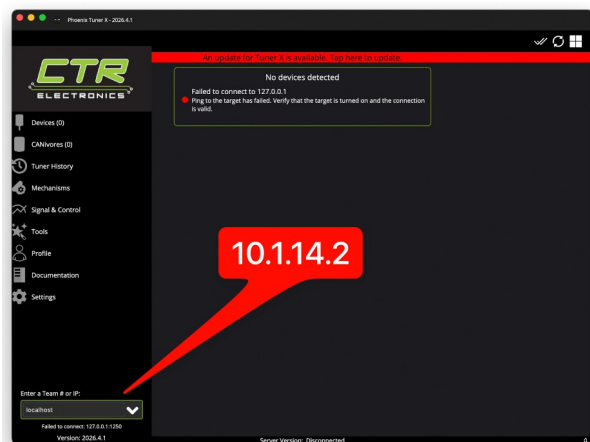
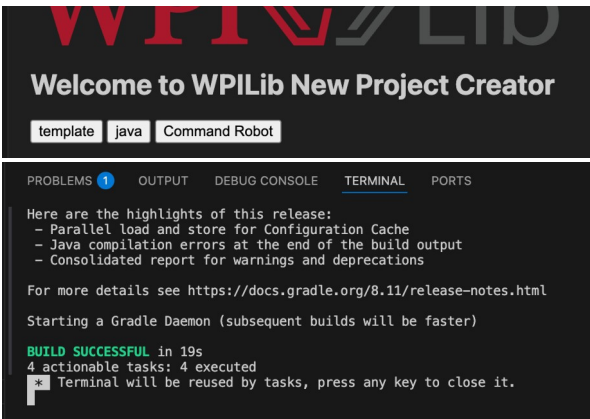
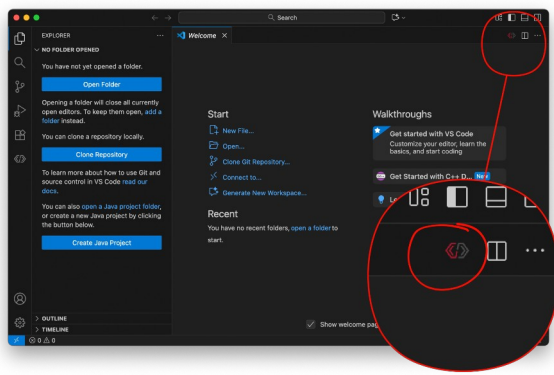
WPILib 2026 was the core install and main software. It bundles its own VScode, which I had initially told it to use my own vscode, but this messed mine up significantly. I had to uninstall both and reinstall them. It also has a gradle build system, simulation tools, and elastic dashboard. Phoenix Tuner X is CTRE's tool for configuration & testing of the TalonFX, and I read the documentation for it to learn about the layout.

Now, my laptop is a Mac, and several FRC tools are Window-only, so for instance I cannot install NI Game Tools.

I read the driver station key shortcuts. This was very important as it had the safety reading: the Space key emergency stops the robot, even after the driven station window is not active. Enter simply disables it, and combining the three keys: (L/), (I/), & (V/) starts it again. I really made sure I knew this before starting the robot in case something went wrong.

I reviewed a Java cheat sheet, as my skills had waned since AP Comp Sci; I reviewed classes, fields, access modifiers (like private final), constructors, method overriding (@Override), and lambdas. WPILib commands use lambdas such as runOnce(()->...). This was different from anything I had seen in AP Comp Sci so I was sure to review them. I also browsed the WHILib github repository to see how the library is organized. It made later steps, like importing SubsystemBase, make more sense.

To conclude: my individual contribution was learning and downloading software; in fact, I was the only member of my group this week.



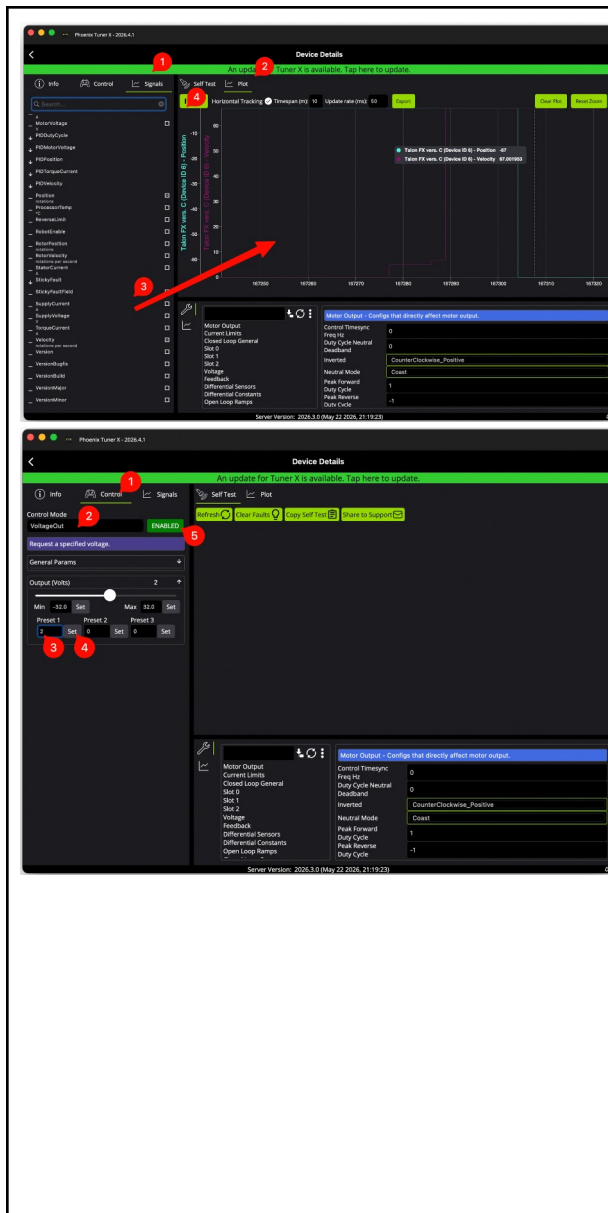
Now, for this section of the project, I was actually coding and getting the test board to work. I opened WPILib and clicked the red command button, circled in the screenshot, and created a new project, using the java template provided. Command robot is the standard FRC structure: hardware, grouped into subsystems, actions, commands that require those subsystems, and this prevents two distinct pieces of code from controlling the same motor at the same time.

I set our team number of 114, which is how the build tools find our roboRIO at 10.1.14.2, and then generated the project.

WPILib does not include CTRE code, so the project could not yet talk to the TalonFX; I copied the Phoenix 6 2026 vendordep URL from CRTE, and then used WPILib manage vendor libraries, and then installed the new library online. However, this caused the build to be unsuccessful, an issue I was stuck on for a while. I realized that I needed an older version of Phoenix 6, and had to manually downgrade to the newest 2025 version (e.g., newest old version). Later, I repeated these steps for DogLog, which resulted in a similar issue. However, I realized very quickly that it was a downgrading issue, and resolved it much faster. I think this is an excellent example of making a mistake, finding an error, fixing it, and becoming faster at it in the future. I also understand now that vendordep urls are just JSON files.

My first goal then was to move the motor without code. I connected to the testboard's ifi and opened Phoenix Tuner X, and connected to Roborio's address, 10.1.14.2. However, this did not work on the first time. Being on the testboard wifi means that I lost internet, and when my laptop reconnected to the school wifi, Tuner lost the RoboRIO. This was a very small issue, but it was infuriatingly hard to find.

In the device details I opened the signals tab, and checked position and velocity. Twisting the motor by hand changed both lines, confirming that the encoder worked. It was also really cool to see, physically, concepts that we learned in calculus (I tried to move the motor in a periodic sine wave, and indeed, the velocity was a sine wave of the same magnitude, but just offset). I also used VoltageOut to give a constant amount of voltage, and it worked perfectly. I just had to keep in mind that I should stay below 6V at all times, no matter what.



To control the testboard, I created Testboard.java and wrote code according to the specifications provided in the SY26 code intro. Similarly, I wrote RobotContainer.java so it constructs a Testboard. For telemetry with DogLog, I installed the vendordep, and used `DogLog.log("Testboard/MotorPosition", motor.getPosition().getValue())` Inside of `periodic()` Which runs every 20ms or so. In elastic dashboard, I added widgets for these and watched them update as the motor moved.

Finally, once the constant voltage code was working very well, I replaced it with a PID controller, following the kP, kI, and kD. It is still limited to 6V. Each loop; proportional; integral; and derivative, computed $\text{error} = \text{target} - \text{position}$. P simply is proportional, I accumulated error over time, and D dampens overshoot.

I had to do a lot of tuning. I started with all gains at 0, raised kP until I saw oscillations, raised kD until it stops. I never had to use kI, as no consistent offset appeared. Oftentimes, a kI term is not needed, unless the system is consistently off.

To conclude, my individual contribution was, well, everything because I was the sole group member. I successfully completed all the projects outlined in the code guide, largely debugging myself. I learned a ton about code and how we can write code to effectively control the robot, and feel confident (and excited!) to work on this in the future.